

MIDDLEWARE PARA COMUNICAÇÃO DISTRIBUÍDA: UM ESTUDO DO NATS

2023008101 - Eleanderson Ambrozio Jeronimo
2023007089 - Gabriel Barbosa Fernandes
2023005075 - Matheus Lázaro de Lima
2020024371 - Nicole Veronica Pinto dos Santos

SEMINÁRIO

Prof. Rafael Frinhani



INSTITUTO DE
MATEMÁTICA E
COMPUTAÇÃO

UNIFEI - Itajubá



Middleware para Comunicação Distribuída: Um Estudo do NATS

1 Introdução

A arquitetura proposta baseia-se no middleware NATS, uma tecnologia de mensageria distribuída orientada ao paradigma *publish/subscribe*. O NATS permite comunicação assíncrona de baixa latência, em que produtores publicam eventos e consumidores inscritos recebem mensagens por meio de subjects.

Como evidência prática, este trabalho apresenta um protótipo com comunicação entre processos distribuídos mediada pelo NATS, implementando um modelo cliente-servidor capaz de demonstrar as operações propostas no seminário, incluindo troca de mensagens, alteração remota de arquivos e execução distribuída de cálculos.

2 Fundamentação Conceitual

2.1 Paradigma Publish/Subscribe

O paradigma *Publish/Subscribe* (Pub/Sub) é um modelo de comunicação assíncrona em que produtores (*publishers*) publicam eventos em canais lógicos, enquanto consumidores (*subscribers*) recebem mensagens associadas aos tópicos de seu interesse. Diferentemente do modelo cliente-servidor tradicional, esse padrão promove desacoplamento entre emissores e receptores. [Microsoft \(2024\)](#)

A comunicação é mediada por um intermediário responsável por distribuir eventos aos assinantes adequados, favorecendo escalabilidade e comunicação reativa em sistemas distribuídos, além de reduzir o acoplamento entre componentes e evitar dependências diretas típicas de modelos síncronos, que podem comprometer a flexibilidade e a resiliência do sistema.

No NATS, esse paradigma é implementado por meio de *subjects*, tópicos hierárquicos utilizados para o roteamento eficiente das mensagens.

2.2 NATS Core

O NATS Core corresponde à camada básica de mensageria do ecossistema NATS e implementa comunicação leve baseada no paradigma *publish/subscribe*. Seu modelo prioriza simplicidade, baixa latência e alto throughput, oferecendo comunicação nativa muitos-para-muitos (M:N) entre processos distribuídos.

Além do paradigma pub/sub discutido anteriormente, o NATS Core incorpora mecanismos como *request-reply*, para interações do tipo requisição-resposta, e *queue groups*, que permitem distribuir mensagens entre consumidores e balancear carga de processamento.

No que se refere à confiabilidade, o NATS Core opera com semântica *at-most-once*, em que a mensagem é enviada sem

persistência nativa ou retransmissão automática, priorizando desempenho e caracterizando o modelo básico de entrega da plataforma.

2.3 JetStream

Complementando o modelo básico do NATS Core, o JetStream introduz persistência e streaming, ampliando as garantias de entrega da plataforma.

Por meio de *streams* e *consumers*, permite armazenamento de mensagens, replay de eventos e desacoplamento temporal entre produtores e consumidores, possibilitando que mensagens sejam consumidas mesmo após sua publicação. Além disso, suporta semânticas de entrega *at-least-once* e cenários *exactly-once* baseados em confirmações de processamento.

Segundo a documentação oficial do NATS, o JetStream também suporta replicação baseada em quorum via algoritmo RAFT, fornecendo tolerância a falhas e alta disponibilidade em ambientes distribuídos [NATS Authors \(2024b\)](#).

3 Arquitetura da Tecnologia NATS

3.1 Componentes

O ecossistema NATS é estruturado a partir de três componentes centrais:

- **NATS Server (*nats-server*):** núcleo responsável por gerenciar conexões, realizar o roteamento das mensagens e manter o estado das assinaturas ativas. Sua arquitetura é orientada a baixa latência e processamento eficiente de eventos.
- **Clients (*Publishers* e *Subscribers*):** aplicações que interagem com o servidor publicando ou consumindo mensagens. No protótipo desenvolvido, o *publisher* representa o serviço emissor de eventos, enquanto os *subscribers* representam os consumidores dessas respostas.
- **Subjects:** tópicos hierárquicos utilizados para endereçamento e roteamento das mensagens (por exemplo, *investimentos.acoes.petr4*). A entrega é baseada no interesse dos assinantes registrados nesses subjects.

3.2 Modelo de Comunicação

O middleware adota um modelo de comunicação assíncrono baseado em troca de mensagens.

- **Publish-Subscribe (1:N):** mensagens publicadas em um *subject* são entregues aos assinantes interessados e ativos no momento da publicação, caracterizando o modelo básico de comunicação do NATS.

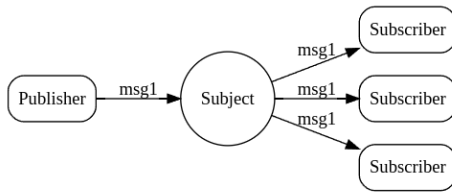


Figura 1: Modelo Publish-Subscribe no NATS.

- **Request-Reply (1:1):** padrão construído sobre pub/sub em que um cliente envia uma requisição e recebe resposta por meio de um *reply subject*, viabilizando comunicação orientada a serviços.

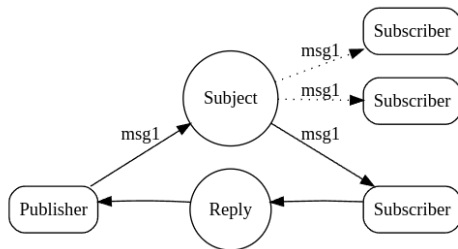


Figura 2: Modelo Request Reply.

- **Queue Groups:** múltiplos consumidores compartilham uma mesma assinatura, mas cada mensagem é processada por apenas um deles, permitindo balanceamento de carga e escalabilidade horizontal.

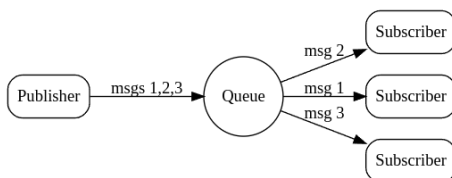


Figura 3: Modelo Queue Groups.

3.3 Subjects

Os *subjects* são os identificadores utilizados pelo NATS para organizar e direcionar a comunicação entre clientes, funcionando como canais lógicos de publicação e subscrição. Esse modelo garante transparência de localidade, já que produtores e consumidores não precisam conhecer a localização uns dos outros.

Eles podem ser estruturados de forma hierárquica por meio do caractere `.` (*ponto*), permitindo organizar tópicos do mais genérico ao mais específico, como em `orders.created`. Essa hierarquia é essencial para o roteamento eficiente das mensagens.

O NATS também suporta *wildcards* em subscrições: `*` (*asterisco*), que representa exatamente um nível da hierarquia, e

`>` (*maior que*), que corresponde a múltiplos níveis subsequentes. Essa flexibilidade permite implementar estratégias como filtragem de mensagens, controle de acesso e otimização do roteamento.

3.4 Roteamento das mensagens

O NATS adota um modelo de roteamento baseado em interesse (*interest-based routing*), no qual os servidores mantêm um grafo dinâmico de *subscriptions* NATS Authors (2026). Esse grafo representa os *subjects* de interesse dos clientes e é propagado entre os nós do cluster. Quando uma mensagem é publicada, ela é encaminhada apenas para os servidores que possuem assinantes interessados no *subject* correspondente, evitando *broadcast* desnecessário e aumentando a eficiência do sistema.

4 Arquitetura e Implementação do Protótipo

4.1 Escopo e Arquitetura

Nesta etapa foi desenvolvido um protótipo utilizando o middleware NATS, com o objetivo de demonstrar a comunicação distribuída entre processos executando em máquinas físicas distintas. O protótipo segue o paradigma de mensageria *publish/subscribe*, permitindo que um cliente envie solicitações e receba respostas processadas por um servidor.

A comunicação entre cliente e servidor ocorre por meio de *subjects* do NATS, garantindo desacoplamento entre os processos e permitindo escalabilidade para múltiplos clientes ou múltiplos serviços.

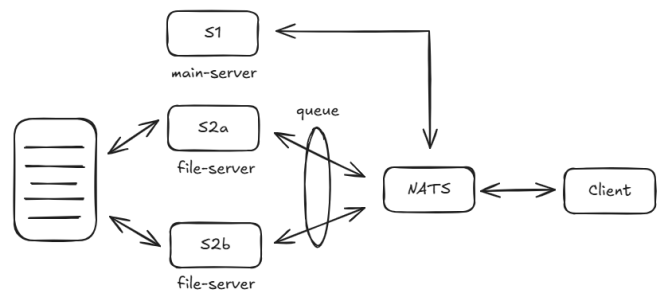


Figura 4: Diagrama da Arquitetura do Protótipo.

4.2 Operações Implementadas

1. **Resposta a mensagem de texto:** O cliente publica uma requisição textual em um *subject* dedicado, que é consumida pelo serviço principal S1. O S1 realiza o parsing da mensagem, processa o conteúdo e retorna uma resposta ao cliente utilizando o padrão *request/reply* do NATS, validando a troca de mensagens com baixa latência.
2. **Alteração de arquivo no servidor:** O cliente envia uma requisição contendo os dados a serem persistidos diretamente ao serviço de arquivos por meio de um *subject* específico. O processamento é realizado por uma das instâncias S2a ou S2b, configuradas em *queue group*, garantindo balanceamento de carga entre os servidores. A

instância selecionada executa a escrita/atualização do arquivo local e retorna ao cliente o status da operação utilizando o mecanismo de request/reply do NATS.

3. **Cálculo de Funções:** O cliente solicita a execução de uma função matemática enviando parâmetros e o tipo de operação desejada. O serviço S1 interpreta a requisição, executa o cálculo e retorna o resultado ao cliente via NATS, utilizando request/reply. Essa operação evidencia o uso do middleware para processamento remoto orientado a mensagens.

4.3 Configuração do Ambiente

Para a execução do protótipo, foi necessária a configuração do servidor NATS e das dependências da linguagem escolhida.

Inicialmente, o servidor NATS foi executado na máquina responsável pelo broker utilizando o comando: `nats-server`.

Em seguida, cliente e servidor foram executados em máquinas físicas distintas, conectadas à mesma rede, configurando o endereço IP do servidor NATS para permitir comunicação remota.

Além disso, foi utilizada a biblioteca oficial do NATS para a linguagem escolhida, responsável por estabelecer conexão, publicar mensagens e realizar subscriptions nos subjects definidos.

5 Evidências Técnicas

5.1 Fluxo de Comunicação

O fluxo de comunicação do protótipo ocorre de forma indireta e desacoplada, sendo mediado pelo middleware NATS, que atua como broker central responsável pelo roteamento das mensagens. O cliente envia requisições publicando mensagens em subjects específicos, e o NATS encaminha essas mensagens aos serviços inscritos nos respectivos canais.

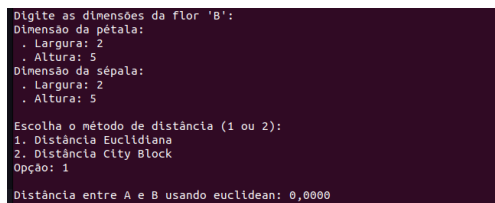
Para operações de texto e cálculo, as mensagens são encaminhadas diretamente ao serviço principal S1 (main-server), que processa a solicitação e retorna uma resposta ao cliente utilizando o mecanismo de request/reply do NATS. Dessa forma, o cliente recebe a resposta sem necessidade de conexão direta com o servidor.

Para operações envolvendo manipulação de arquivos, o cliente publica a requisição diretamente no subject correspondente ao serviço de arquivos. Essa requisição é entregue a apenas uma das instâncias do serviço (S2a ou S2b) devido ao uso de queue group, permitindo balanceamento de carga entre servidores e evitando processamento duplicado. A instância selecionada realiza a operação no armazenamento e retorna o resultado ao cliente por meio do mecanismo de request/reply do NATS. Esse modelo promove maior desacoplamento entre os componentes, além de permitir escalabilidade horizontal e maior tolerância a falhas no serviço de arquivos.

5.2 Resultados

Os testes realizados demonstraram que o protótipo implementado com NATS foi capaz de executar corretamente as operações propostas, garantindo comunicação distribuída eficiente entre os componentes. Em todas as requisições enviadas pelo

cliente, o sistema retornou respostas estruturadas em formato JSON, contendo código de status, mensagem descritiva e os dados correspondentes ao processamento.



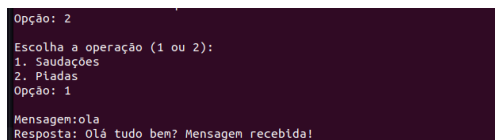
```

Digite as dimensões da flor 'B':
Dimensão da pétala:
. Largura: 2
. Altura: 5
Dimensão da sépala:
. Largura: 2
. Altura: 5

Escolha o método de distância (1 ou 2):
1. Distância Euclidiana
2. Distância City Block
Opção: 1

Distância entre A e B usando euclidean: 0,0000
  
```

Figura 5: Cálculo de funções.



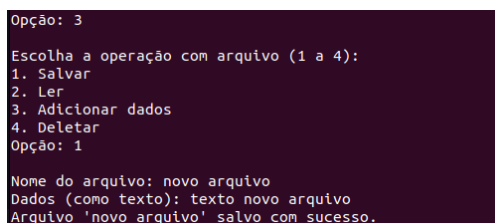
```

Opção: 2

Escolha a operação (1 ou 2):
1. Saudações
2. Piadas
Opção: 1

Mensagem:ola
Resposta: Olá tudo bem? Mensagem recebida!
  
```

Figura 6: Resposta a mensagem de texto.



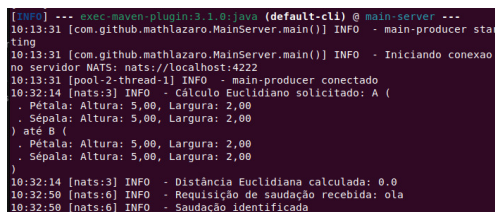
```

Opção: 3

Escolha a operação com arquivo (1 a 4):
1. Salvar
2. Ler
3. Adicionar dados
4. Deletar
Opção: 1

Nome do arquivo: novo arquivo
Dados (como texto): texto novo arquivo
Arquivo 'novo arquivo' salvo com sucesso.
  
```

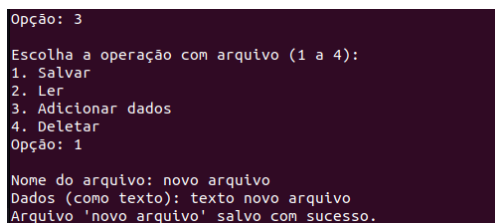
Figura 7: Alteração de arquivo.



```

[INFO] --- exec-maven-plugin:3.1.0:java (default-cli) @ main-server ---
10:13:31 [com.github.mathlazarro.MainServer.main()] INFO - main-producer starting
10:13:31 [com.github.mathlazarro.MainServer.main()] INFO - Iniciando conexão no servidor NATS: nats://localhost:4222
10:13:31 [pool-2-thread-1] INFO - main-producer conectado
10:13:14 [nats:3] INFO - Cálculo Euclidiano solicitado: A (
. Pétala: Altura: 5,00, Largura: 2,00
. Sépala: Altura: 5,00, Largura: 2,00
) até B (
. Pétala: Altura: 5,00, Largura: 2,00
. Sépala: Altura: 5,00, Largura: 2,00
)
10:13:14 [nats:3] INFO - Distância Euclidiana calculada: 0.0
10:13:50 [nats:6] INFO - Requisição de saudação recebida: ola
10:13:50 [nats:6] INFO - Saudação identificada
  
```

Figura 8: Servidor resposta a mensagem.



```

Opção: 3

Escolha a operação com arquivo (1 a 4):
1. Salvar
2. Ler
3. Adicionar dados
4. Deletar
Opção: 1

Nome do arquivo: novo arquivo
Dados (como texto): texto novo arquivo
Arquivo 'novo arquivo' salvo com sucesso.
  
```

Figura 9: Servidor alteração de arquivo.

Conforme evidenciado na execução, a operação de envio de mensagem, alteração de arquivo ou cálculo matemático foi concluída com sucesso, retornando mensagens de confirmação como "Operação realizada com sucesso" e status 200, indicando que o fluxo de comunicação baseado em request/reply funcionou corretamente. Dessa forma, os resultados validam que o NATS realizou o roteamento das mensagens de forma adequada e que os serviços distribuídos foram capazes de processar e responder às solicitações conforme o esperado.

6 Uso Prático e Ecossistema

6.1 Linguagens e Bibliotecas

O NATS é agnóstico de linguagem, com suporte oficial e comunitário para dezenas de linguagens, incluindo Go, Python, Java, C/C++, C#, e JavaScript.

6.2 Ferramentas Disponíveis

Para o gerenciamento e monitoramento do cluster, o ecossistema disponibiliza ferramentas oficiais:

- **NATS CLI:** administração, testes de publicação/assinatura e gerenciamento do JetStream;
- **NATS Server:** servidor distribuído em binário único, adequado para ambientes Docker e Kubernetes.

6.3 Aplicações Reais

Casos reais de adoção mostram o uso do NATS em aplicações distribuídas com requisitos de desempenho e escalabilidade.

- **Finanças e Pagamentos:** uso em mensageria transacional, processamento de eventos e distribuição de dados em tempo real para sistemas financeiros.
- **IoT e Telemetria:** uso em coleta de métricas, comunicação entre dispositivos distribuídos e arquiteturas edge. Empresas como Siemens aparecem entre casos associados ao ecossistema NATS.
- **Microserviços e Alta Concorrência:** uso como backbone de comunicação entre serviços distribuídos, com aplicações reportadas em empresas como Capital One, Tinder e outras organizações do ecossistema cloud-native.

–

7 Vantagens

O NATS, por ser escrito em Go, oferece altíssimo desempenho, roteando milhões de mensagens por segundo com latência de microssegundos, próximo ao limite da rede física. É leve e simples, com um único binário sem dependências externas, baixo consumo de RAM e deploy facilitado. Além disso, sua arquitetura em cluster permite auto-descoberta, escalabilidade e auto-reparo, reconfigurando a rede automaticamente em caso de falha de um nó, garantindo alta disponibilidade sem interrupções.

7.1 Limitações

O NATS possui limitações como o tamanho restrito das mensagens (1 MB por padrão), sendo mais adequado para mensagens curtas. Também não oferece lógica complexa de roteamento como outros middlewares, utilizando apenas o casamento de strings nos subjects. Além disso, na versão básica (Core), não há persistência nativa — mensagens sem consumidores ativos são descartadas, sendo necessário usar o módulo JetStream para persistência.

7.2 Comparação com Tecnologias Correlatas

A Tabela apresenta uma comparação entre o NATS e outras soluções de mensageria amplamente utilizadas, com base em características discutidas por Sharvari & Nag K. (2019) e na documentação oficial do NATS NATS Authors (2024a).

Tabela 1: Comparação entre middlewares de mensageria

Critério	NATS	RabbitMQ	Kafka
Latência	Muito baixa	Média/baixa	Média
Persistência nativa	Via JetStream	Sim	Sim
Roteamento complexo	Básico	Avançado	Médio
Escalabilidade	Alta	Média/alta	Alta
Complexidade operacional	Baixa	Média/alta	Alta

7.3 Cenários Adequados

O NATS é indicado para sistemas financeiros e de alta frequência, onde o tempo mínimo de resposta é essencial para distribuir alertas em tempo real. Também é adequado para arquiteturas de microserviços reativos, atuando como infraestrutura de troca eficiente de mensagens entre serviços, além de aplicações de IoT e telemetria, suportando grandes volumes de pequenos pacotes de dados enviados por sensores e dispositivos conectados.

7.4 Cenários Não Recomendados

Não é indicado para transferência de arquivos grandes devido ao limite de tamanho das mensagens, nem para sistemas com transações complexas (ACID), que exigem garantias rigorosas de consistência e rollback. Também não é a melhor escolha para filas de processamento pesado e tarefas de longa duração, onde soluções especializadas em gerenciamento de tarefas podem ser mais adequadas.

8 Links do Projeto

- Repositório do código-fonte: <https://github.com/mathLazaro/prototipo-nats>

Referências

- Microsoft (2024). Padrão publisher-subscriber. Acesso em: 28 abr. 2026.
- NATS Authors (2024a). Compare nats with kafka, rabbitmq and others. Acesso em: 29 abr. 2026.
- NATS Authors (2024b). Jetstream concepts and architecture. Acesso em: 29 abr. 2026.
- NATS Authors (2026). Nats faq.
- Sharvari, T. & Nag K., S. (2019). A study on modern messaging systems: Kafka, rabbitmq and nats streaming. *arXiv preprint arXiv:1912.03715*.

